Graphic Era deemed to be University DEHRADUN

# PROJECT AND TEAM INFORMATION

## Project Title

**Static Code Analyzer for a C-like Language using CFG-Based Data Flow Analysis**

## Student/Team Information

| | |
|---|---|
| *Team Name:*<br>*ALGO INVENTORS* | |
| **Team member 1 (Team Lead)**<br>Name : Akshay Raj<br>Student id : 230211569<br>Email id : akshayraj2502@gmail.com | |

| | |
|---|---|
| **Team member 2**<br>Name : Aryan Kumar<br>Student ID : 23021658<br>Email : aryan133067@gmail.com | |

| | |
|---|---|
| **Team member 3**<br>Name : Gyanendra Bhardwaj<br>Student id : 230211195<br>Email id : gyan94172@gmail.com | |
| Team member 4<br>Name :Amogh Bhatt<br>Student id : 23021909<br>Email id : amogh23bhatt@gmail.com | |

# PROJECT PROGRESS DESCRIPTION (35 pts)

## Project Abstract

This project describes the design and implementation of a Static Code Analyzer for a C-like language using Control Flow Graph (CFG) based Data Flow Analysis. The main goal of this is to analyze source code without running it, so we can find all the types of errors, improve code quality, and understand how the program behaves.

Firstly, the input code is broken into small pieces called tokens (lexical analysis). Then, a parser uses these tokens to create an Abstract Syntax Tree (AST), which shows the structure of the program. After that, semantic analysis is done to find errors like undeclared variables, scope problems, and wrong use of variables.

Once the semantic checks are complete, we transform the AST into a Control Flow Graph (CFG). This graph shows the flow of execution in the program through basic blocks. We then apply data flow analysis techniques on the CFG to track variable usage and identify problems like undefined variables and improper assignments.

The project also offers an optional visualization feature using Graphviz and we analyse how these modern tools like sonarqube works that it analyzes the source code without running it. This feature allows users to create graphical representations of the CFG to better understand program flow.

So, this project combines compiler design principles with static analysis techniques to create a tool that improves program correctness and reliability.

## Updated Project Approach and Architecture

We have done this project in such a way  that's similar to a compiler. We have broken the system down into parts and each part does a specific job.

First we give the system the source code that we want it to work with. The lexer, which we made using ANTLR, reads the code. Break it down into small pieces called tokens. These tokens are then passed to the parser, which creates a map of the code structure called an Abstract Syntax Tree or AST.

After the parser is done we check the code to make sure it is correct. We look for mistakes like variables that are not declared and identifiers that are not used correctly. Once we are sure the code is okay we create a Control Flow Graph or CFG which shows how the program will run.

We use the CFG to see how the variables are used and to find problems like variables that are not defined. We also have a way to visualize the CFG using Graphviz, which makes it easy for users to see how the program works.

Our project is written in C++. We use ANTLR to make the lexer and parser. We use Graphviz to make the visualization. The different parts of the system talk to each other by passing around data like tokens and CFG.

Overall we tried to make the system simple and easy to work with so it will be easy to add features later on. Our project follows an approach and this modular approach is what makes it easy to understand and work with. The system is designed to be easy to extend. This is why we chose a modular approach for our project.

## Tasks Completed

| Task Completed | Team Member |
|---|---|
| Project setup and repository creation | Akshay Raj |
| Environment setup (ANTLR, Graphviz, VS Code) | Aryan Kumar |
| Lexer implementation using ANTLR | Gyanendra Bhardwaj |
| Basic semantic analysis implementation | Akshay Raj |
| Control Flow Graph (CFG) generation | Amogh Bhatt |
| Testing with sample input (test.c) | Amogh Bhatt |

## Challenges/Roadblocks

Building this project brought some challenges along the way.

Understanding ANTLR took time, especially learning how it is used to create the lexer and parser. Writing grammar rules for a C-like language was difficult in the beginning. To improve, we are studying examples and slowly improving the grammar step by step.

Setting up the project environment was also challenging. Tools like ANTLR and Graphviz needed proper installation and configuration. We carefully followed each step and checked everything in VS Code. We solved this by installing tools one by one and verifying that all paths and settings were correct.

Understanding the Abstract Syntax Tree (AST) and Control Flow Graph (CFG) also took time. These are complex concepts and not easy to implement. Instead of doing everything at once, we are working step by step. We complete one part first and then move to the next, which makes the process easier.

Data flow analysis is also challenging because it requires tracking variables throughout the program. To solve this, we are first learning the basic concepts and then applying simple methods.

Debugging the compiler is difficult because one small mistake can affect the whole system. To handle this, we test each module separately to find and fix errors.

We are overcoming these challenges through continuous learning and practice. By working step by step and improving gradually, we are making steady progress in the project.

## Tasks Pending

| Task Pending | Team Member (to complete the task) |
|---|---|
| 1. Improvement of the semantic analysis to allow the implementation of features like type checking | Amogh Bhatt |
| 2. Completing the implementation of the data flow analysis | Gyanendra Bhardwaj |
| 3. Improvement of the Control Flow Graph (CFG) generation to allow the analysis of more complex code | Aryan Kumar |

| | |
|---|---|
| 4. Development of additional test cases to ensure the validation of different code scenarios | Amogh Bhatt |
| 5. Improvement of the error messages to ensure they are clear and user-friendly | Akshay Raj |
| 6. Improvement of the structure of the code to ensure better modularity | Aryan Kumar |
| 7. Final testing and debugging of the system | Akshay Raj |

## Project Outcome/Deliverables

The major achievement of the project is the development of a Static Code Analyzer for a C-like language. The system is capable of taking the source code as input and analyzing the code without executing the program.

The project has different modules, such as a lexer, parser, and semantic analyzer, which work together to process the code step by step. The project has also achieved the development of an Abstract Syntax Tree (AST) to represent the structure of the code.

Another major achievement of the project is the development of a Control Flow Graph (CFG), which represents the execution flow of the code. On the CFG, data flow analysis is conducted to identify variables and errors such as undefined variables.

The project has achieved the development of the Control Flow Graph (CFG) and its visualization, which is helpful in understanding the flow of the code easily by using Graphviz. The project has achieved all the requirements to be a complete project, thus helping in improving the quality of the code by identifying errors.

## Progress Overview

The project is currently under good progress. The initial setup, which includes installing tools such as ANTLR and Graphviz, as well as setting up the environment inside VS Code, has been completed successfully.

The setup for the lexer and parser using ANTLR has been completed. The project structure and grammar rule understanding are mostly complete. The initial setup for the project, which includes setting up the lexer and parser, as well as understanding the grammar rules, is mostly complete.

The development for the Abstract Syntax Tree and semantic analysis is partially complete. The error checking is partially working, but this is still a work in progress. The development for the Abstract Syntax Tree and semantic analysis is partially complete, as well as the error checking. The development for the Control Flow Graph and data flow analysis is not yet complete, but this is slightly behind schedule. The development for the Control Flow Graph and data flow analysis is more complex, so this requires more understanding. The visualization part, which uses Graphviz, is planned

and will be added later, but this will be done once the Control Flow Graph is completed. The project is progressing well, as most parts of the project are still under development

## Codebase Information

The project code is hosted on GitHub.

1. Repository Link: https://github.com/Akshay-X-Raj/Static-Code-Analyzer-for-a-C-like-Language-using-CFG-Based-Data-Flow-Analysis
2. Branch Used: main

In the repository, the necessary components of the project have been included:

1. src/ – The actual code is present here.
2. grammar/ – The ANTLR grammar file is present here (MiniC.g4).
3. tests/samples/ – Test files are present here (test.c).
4. build/ – Compiled output files.
5. cfg.dot & cfg_visual.png – CFG output & visualization.

Some of the commits made to the repository:

1. Initial commit – Basic project structure.
2. README update – Project description & instructions.
3. Analyzer with output (test.c) – Project implementation with test file output.

The project is updated regularly to reflect the changes made to the project to ensure improvements

## Testing and Validation Status (2 pts)

(Provide information about any tests conducted)

| TEST TYPE | STATUS (PASS/FAIL) | NOTES |
|---|---|---|
| Lexical analysis test | PASS | Tokens are generated correctly |
| Parsing Test | PASS | AST is created properly |
| Semantic Analysis Test | PASS | Basic errors are detected |
| CFG Generation Test | PASS | CFG is generated correctly |
| Data Flow Analysis Test | IN PROGRESS | Not fully completed yet |
| Visualization Test (Graphviz) | PASS | Graph is shown correctly |
| Sample Code Testing | PASS | Works well with sample code |

## Deliverables Progress

The deliverables of the project are at various stages of completion.

1. **Lexer and Parser**: Completed. The system can successfully convert the input code to tokens and generate the Abstract Syntax Tree.
2. **Abstract Syntax Tree (AST):** Completed. The structure of the program is represented.
3. Semantic Analysis: In progress. Basic features like checking for undeclared variables are working; however, more features are still being added.
4. **Control Flow Gr**aph (CFG): Completed. The system can generate a CFG from the input code.
5. **Data Flow Analysis:** In progress. The basic structure of this feature is ready; however, more features are still being added.
6. **CFG Visualization (Graphviz):** Completed. The system can generate and display a CFG in graphical form.
7. **Testing with Sample Code:** Completed. The system can run with sample codes like 'test.c' and can generate output.